Received [date], accepted [date], date of publication [date], date of current version [date].
Digital Object Identifier [DOI]

GitPulse: A Data Engineering Pipeline for Repository Health Analytics

Akhila Mulukuta • P.Donica Mrinmayi

[Department, Institution, City, Postal Code, Country]

Corresponding author: [Author Name] ([email address])

This work was supported in part by [funding source / grant number].

ABSTRACT This tutorial paper presents GitPulse, an end-to-end data engineering pipeline that turns raw GitHub activity into a single, explainable Repository Health Score. We describe how each stage of a modern data engineering platform (data connection, ingestion, transformation and processing, storage, metrics, visualization and orchestration) is realized with lightweight, open-source tools: the GitHub REST API, a paginated ingestion layer, a pandas-based transformation layer, a SQLite store, a metrics layer and a Streamlit and Plotly dashboard. We then formalize a health model that combines commit frequency, active contributors, pull request (PR) closing speed and issue resolution speed into a score between 0 and 100, using reference-value normalization and automatic re-weighting when a metric has no data. The results of our analysis show that a small, modular pipeline can deliver actionable repository-level insight, but that it also exposes concrete gaps: pagination-capped sampling, identity ambiguity in commit authorship, schema collisions across forks, and the absence of incremental loading. We discuss these gaps and challenges and propose recommendations for moving from a single-user analytics tool toward a scalable, incremental and validated repository analytics platform.

INDEX TERMS Data engineering, GitHub analytics, open-source software, repository health, data pipelines, software analytics, tutorial.

I. INTRODUCTION

Open-source software now underpins most commercial and scientific computing, and the question “is this repository healthy?” has become central to dependency selection, maintainer planning and funding decisions. GitHub hosts an enormous volume of public activity (commits, pull requests, issues, reviews and discussions) that is exposed through a documented REST API [2]. Yet the raw API output is paginated, nested, rate-limited and inconsistent across endpoints, so turning it into decision-ready information is a data engineering problem before it is an analytics problem.

The popularity signals most developers rely on, such as stars and forks, are weak proxies for project vitality [10]. Studies of why open-source projects fail point instead to sustained contributor activity and responsiveness [11], and the CHAOSS community has proposed standard metrics for community health [5]. Measuring these signals reliably requires a pipeline that connects to the source, ingests data within a bounded window, cleans and stores it, computes metrics and presents them to the user. These are the same building blocks that recent tutorial work identifies for general data engineering platforms [1].

GitPulse is a compact implementation of that pipeline. Given any public repository, it fetches recent commits, PRs and issues, loads them into a relational store, computes four activity and responsiveness metrics and combines them into a Repository Health Score from 0 to 100, which is shown in an interactive dashboard. Because every stage is small enough to read in a single sitting, GitPulse is also a useful teaching vehicle for the data engineering concepts of connection, ingestion, transformation, storage, orchestration and visualization.

A. RELATED WORKS

Mining software repositories has a long history. GHTorrent [3] provided a large, queryable mirror of GitHub event data and made repository-scale studies possible, while Kalliamvakou et al. [4] catalogued the perils of mining GitHub, including inactive repositories, personal projects and the mismatch between merged and closed pull requests. Jansen [9] argued that project health must be measured at several levels, from the single project up to its surrounding ecosystem. The CHAOSS project [5] standardizes metrics such as contributor counts, issue response time and change-request closure, which directly inspire the metrics used here.

On the data engineering side, surveys describe the modular structure of data platforms and the frameworks available for each module: Kafka for ingestion [14], Spark [12] and Flink [13] for processing, relational and NoSQL systems for storage, and orchestration engines for workflow control [1]. Sculley et al. [15] warned that glue code and pipeline sprawl create hidden technical debt in data-driven systems, a concern that motivates GitPulse’s deliberately small and modular design.

However, these contributions leave three gaps. Repository-mining datasets are typically large offline dumps rather than on-demand pipelines. Health metrics are usually reported as separate indicators rather than a single explainable score with transparent weights. And data engineering tutorials focus on large-scale frameworks, with little guidance on how the same

stage-by-stage separation applies to a small analytics application. GitPulse addresses all three.

B. OVERVIEW AND TUTORIAL OBJECTIVES

Fig. 1 shows the six modules of the GitPulse platform, which mirror the general data engineering architecture of [1]: Data Connect, Data Ingestion, Data Processing and Analytics, Data Storage, Data Monitoring and Visualization, and Data Management and Orchestration. The connections between modules are linear in the current implementation, but each module can be replaced independently.

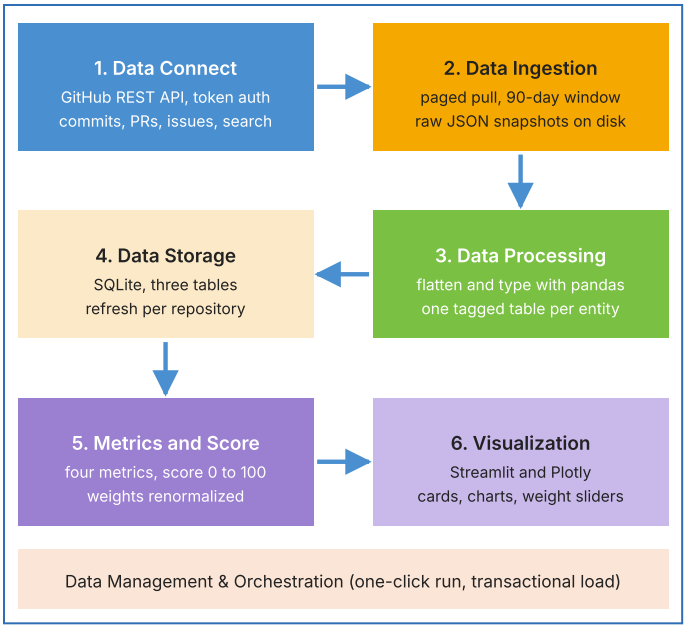


FIGURE 1. High-level illustration of the individual modules of the GitPulse platform. Data flows from the GitHub API through ingestion, processing and storage to the metrics layer, and the resulting Health Score is shown in the dashboard; orchestration spans all modules.

The main contributions of this paper are as follows.

- A stage-by-stage description of a working GitHub analytics pipeline, mapped to the modules of a general data engineering platform.
- A formal, weighted and explainable Repository Health Score with reference-value normalization and automatic weight renormalization for missing data.
- A critical gap analysis that identifies the sampling, identity, schema and scalability limitations of the current design.
- A set of challenges and recommendations for evolving the pipeline toward incremental loading, validation and scalable orchestration.

The remainder of the paper is organized as follows. Section II introduces data connection and the GitHub data sources. Section III presents ingestion. Section IV discusses transformation and processing. Section V describes storage. Section VI defines the metrics and the Health Score. Section VII covers visualization and monitoring. Section VIII covers orchestration and management. Section IX presents the evaluation approach and a worked example. Section X provides the gap analysis, Section XI the challenges, Section XII future directions, and Section XIII concludes the paper.

II. DATA CONNECTION FRAMEWORKS

The data connection module is the trigger that links the pipeline to an external source, and in GitPulse that source is the GitHub REST API [2]. REST is stateless and exposes structured resources, which makes it a natural way to import third-party data into an analytics pipeline [1]. GitPulse uses the `requests` library and a small configuration module that reads a personal access token from an environment file and builds the authorization and `Accept` headers shared by every call.

A. AUTHENTICATION AND CONFIGURATION

The token is loaded with `python-dotenv` so that credentials never appear in source code. All endpoints share one base URL and one header dictionary, so a change of token or API version touches a single file. A separate smoke-test script calls the authenticated user endpoint and reports the account name, which verifies the connection before any long-running ingestion starts. Authenticated requests also raise the API rate limit well above the unauthenticated level, which matters because a single repository analysis issues several paginated calls.

B. DATA SOURCES TO CONNECT

GitPulse connects to five endpoint families, summarized in Table 1.

TABLE 1. GitHub endpoints used by GitPulse.

Endpoint	Purpose	Key parameters
`/search/repositories`	Repository discovery by keyword	`q`, `sort=stars`, `per_page`
`/search/repositories` (popular)	Default landing list of well-known repositories	`q=stars:>1000`, `order=desc`
`/repos/{owner}/{repo}/commits`	Commit history	`since`, `per_page=100`
`/repos/{owner}/{repo}/pulls`	Pull requests, all states	`state=all`, `sort=updated`, `direction=desc`
`/repos/{owner}/{repo}/issues`	Issues (also returns PRs)	`state=all`, `since`, `sort=updated`

These map to the three data groups that data engineering platforms typically meet: application and service data (the commits, PRs and issues themselves), user data (author names and logins), and metadata (stars, forks, language and open issue counts shown in the repository cards) [1].

III. DATA INGESTION FRAMEWORKS

The ingestion module moves data from the connection layer into durable files so that later stages never depend on a live API call. In large platforms this role is played by message brokers such as Kafka [14] that decouple producers and consumers.

GitPulse sits at the other end of the scale: it uses synchronous, pull-based, batch ingestion with explicit pagination, which is simple, debuggable and sufficient for a single-user analytics tool.

A. PULL-BASED BATCH INGESTION

In the queuing-versus-streaming taxonomy of [1], GitPulse is neither: it is a bounded batch pull. Each run requests a fixed analysis window of 90 days, writes the raw JSON of each endpoint to a timestamped file under `data/raw/`, and returns the same records to the caller. Keeping the raw files means a failed transformation can be replayed without calling the API again, which is the same replay property that makes event logs valuable in streaming systems [17].

B. PAGINATION AND EARLY STOPPING

A single generic function fetches pages by following the `next` link that GitHub returns in the response headers. It enforces three safeguards: a maximum of three pages per endpoint, a 30-second request timeout and a 0.2-second pause between pages. For endpoints sorted by update time, it also stops early once the oldest record on a page is older than the analysis cutoff, so the pipeline does not download history it will discard.

The three-page cap bounds both runtime and rate-limit consumption, but it has a statistical cost. With 100 records per page, at most 300 records per endpoint are collected, so very active repositories are truncated and their activity is under-counted. We return to this limitation in Section X.

C. ENDPOINT-SPECIFIC HANDLING

- **Commits** are filtered server-side with the `since` parameter.
- **Pull requests** are requested in all states, newest update first, then filtered client-side to keep only those updated inside the window.
- **Issues** share an endpoint with PRs, because GitHub models every PR as an issue. GitPulse removes any record containing a `pull_request` key and logs how many were dropped, which prevents PRs from being counted twice.

D. USE CASES AND REQUIREMENTS

The ingestion layer must satisfy four requirements: bounded latency (an analysis should finish within a user's attention span), bounded API cost, reproducibility from stored raw data, and graceful failure. Non-200 responses are printed with their body and terminate that endpoint's pagination without crashing the application, so a partial result is returned rather than an exception.

IV. DATA PROCESSING AND TRANSFORMATION FRAMEWORKS

The transformation module converts deeply nested API payloads into flat, typed, analysis-ready tables. Large platforms perform this step with distributed engines such as Spark [12] or Flink [13]; GitPulse performs it in memory with pandas [6], which is appropriate because a single repository window holds at most a few hundred records per endpoint.

A. SCHEMA FLATTENING

Each raw record is reduced to the fields the metrics layer needs and tagged with its repository name so that many repositories can share one database. The extraction uses defensive dictionary access, so a missing author or date produces a null value rather than an exception. Table 2 lists the resulting schema.

TABLE 2. Relational schema produced by the transformation and load stages.

Table	Primary key	Columns
`commits`	`sha`	`repo`, `author`, `date`, `message`
`pull_requests`	`id`	`repo`, `number`, `title`, `state`, `created_at`, `closed_at`, `author`
`issues`	`id`	`repo`, `number`, `title`, `state`, `created_at`, `closed_at`, `author`, `comments`

The commit author is taken from the git author name inside the commit object, whereas PR and issue authors are taken from the GitHub account login. These are different identity spaces, which creates the attribution problem discussed in Section X.

B. TIME HANDLING

Timestamps are stored as ISO 8601 text and parsed at analysis time with `utc=True` and `errors="coerce"`, so malformed values become missing values that are dropped instead of failing the whole computation. This keeps every comparison against the 30-day and 90-day cutoffs timezone-safe.

C. BATCH VERSUS STREAM SEMANTICS

Because the pipeline processes a bounded window on demand, it uses batch semantics, in which results are slower to arrive but exact for the window [1]. The event-time versus processing-time distinction is nevertheless relevant: all metrics are computed from event timestamps (when a commit was authored or a PR was closed), not from the time the pipeline ran, which makes results reproducible for a fixed dataset.

V. DATA STORAGE FRAMEWORKS

The storage module persists the transformed tables so that metrics and dashboards can be recomputed without the network. GitPulse uses SQLite [7], an embedded relational database that requires no server, stores everything in a single file and offers ACID transactions, which suits a single-user application.

A. WHY A RELATIONAL STORE

The data is structured, the access pattern is read-heavy filtering by repository, and the volume is small. In the taxonomy of [1], this makes a relational database preferable to a data lake or a NoSQL store, which are better suited to unstructured or very high-volume data. The schema file is executed with `CREATE TABLE IF NOT EXISTS` on every connection, so the database initializes itself on first use.

B. REFRESH-BY-REPLACEMENT LOADING

To avoid duplicates, the loader removes every existing row for the selected repository and then inserts the freshly transformed rows inside a single transaction. If any insert fails, the transaction is rolled back and the previous data remains intact. This pattern is idempotent: re-analyzing a repository always produces the same table contents for the same input.

C. PRACTICAL ASPECTS

SQLite serializes writers, so it does not support concurrent multi-user analysis. Moreover, the commit table uses the commit hash alone as primary key. A commit shared by a repository and its fork would collide, and the second load would fail. A composite key of repository and hash removes the problem. Both points are revisited in Section X.

VI. METRICS AND REPOSITORY HEALTH SCORE

The analytics module turns stored rows into four metrics and combines them into one score between 0 and 100. This is the descriptive-analytics layer of the pipeline in the sense of [1]: it explains what has happened in the recent past using statistics over real data. Every metric is a single function over the database, so each can be tested and reused independently of the dashboard.

A. BASE METRICS

TABLE 3. Metrics computed by GitPulse.

Metric	Definition	Window	Missing data
Commit frequency	Mean commits per week over the 4 most recent weeks that contain commits	up to 90 days stored	No commits: undefined
Active contributors	Unique commit author names	last 90 days	No commits: 0
PR closing time	Mean of (closed_at − created_at) in hours, over PRs closed in the window	last 90 days	None closed: undefined
Issue resolution time	Mean of (closed_at − created_at) in hours, over issues closed in the window	last 90 days	None closed: undefined
Open PRs / open issues	Count of rows with state open	current snapshot	0

A subtle but important design choice is that resolution time is measured from creation to closure for items that were closed recently. An item created years ago but closed last week therefore contributes its full lifetime, which penalizes repositories that are clearing old backlog. This is a defensible choice for a responsiveness indicator, and it is stated explicitly on the dashboard so users can interpret it.

B. NORMALIZATION

The four metrics have different units and directions, so each is mapped to a score between 0 and 1. Activity metrics (higher is better) use reference-max normalization, and time metrics (lower is better) use inverse normalization against a target:

$$s_{\text{act}}(x) = \min\left(\frac{\max(x, 0)}{x_{\text{ref}}}, 1\right) \tag{1}$$

$$s_{\text{time}}(t) = \begin{cases} 1, & t \le 0 \\ \min\left(\frac{t_{\text{target}}}{t}, 1\right), & t > 0 \end{cases} \tag{2}$$

TABLE 4. Reference values used by the score.

Component	Symbol	Reference	Direction
Commit frequency	x_ref	20 commits per week	higher is better
Active contributors	x_ref	15 contributors	higher is better
PR closing time	t_target	72 hours	lower is better
Issue resolution time	t_target	168 hours	lower is better

A metric that reaches its reference value, or beats its target, earns a full component score. The reference values are design parameters rather than universal constants, and they should be tuned to the population of repositories being compared.

C. WEIGHTED AGGREGATION WITH MISSING-DATA HANDLING

Let s_i be the component score for metric i and w_i its weight. Only components with data (s_i not null) and a positive weight are kept, and the remaining weights are renormalized so that they still sum to one:

$$H = 100 \cdot \frac{\sum_{i \in A} w_i \, s_i}{\sum_{i \in A} w_i} \tag{3}$$

where A is the set of available components. With the default weights of 0.25 per component, a repository with no closed issues is scored on the other three components, each with weight one third, instead of being punished with a zero. If no component is available the score is undefined and the dashboard shows N/A. The sidebar sliders let users change the weights interactively, so the score is explainable and adjustable rather than a black box.

D. WORKED EXAMPLE

Consider a hypothetical repository with 10 commits per week, 6 active contributors, an average PR closing time of 36 hours and an average issue resolution time of 336 hours. The component scores are 10/20 = 0.50, 6/15 = 0.40, min(72/36, 1) = 1.00 and 168/336 = 0.50. With equal weights, $H = 100 \times 0.25 \times (0.50 + 0.40 + 1.00 + 0.50) = 60.0$. If this repository had no closed issues, $H = 100 \times (0.50 + 0.40 + 1.00) / 3 = 63.3$. The example is illustrative, not a measured result.

The score is a composite indicator. It should always be read together with its components, which GitPulse shows as an individual bar chart, because a high value on one component can mask weakness on another.

VII. DATA MONITORING AND VISUALIZATION FRAMEWORKS

The visualization module is the user-facing end of the pipeline, where metrics become decisions. GitPulse uses Streamlit [8] for the application shell and Plotly [16] for charts, the same family of Python tools that the tutorial in [1] lists for data science web applications. Both let the whole dashboard be written as one Python script without a separate front-end project.

A. DASHBOARD COMPONENTS

The interface has four parts: a discovery view, a methodology panel, an analysis trigger and a results view.

- **Discovery.** A keyword search and a default list of popular repositories are shown as cards with stars, forks, language

and description. Selecting a card stores the repository in session state.

- **Methodology.** An expandable panel documents the pipeline, the reference values, the analysis windows and the creation-to-closure rule, so the score is never presented without its definition.
- **Analysis.** A single button runs ingestion, transformation and loading, then refreshes the cached database tables.
- **Results.** The page shows the Health Score, 30-day commits, 90-day active contributors, average PR closing time, a component bar chart, a weekly commit line chart, PR and issue resolution-time histograms, and open and closed counts.

B. INTERACTIVITY AND CACHING

Database tables are cached for five minutes and popular repositories for ten, which reduces both database reads and API calls on reruns. Cache invalidation after each analysis ensures the user always sees fresh results. Session state preserves the selected repository across Streamlit's rerun model, and sidebar sliders recompute the score immediately when weights change.

C. DESIGN PRINCIPLES

Three principles guide the interface. Show the components next to the composite score. Show the definition next to every metric. Show missing data as N/A instead of zero, so absence of evidence is never displayed as bad health.

VIII. DATA ORCHESTRATION AND MANAGEMENT

The orchestration module coordinates the stages and decides when each runs. In production data platforms this is the role of workflow engines such as Apache Airflow, which represent pipelines as directed acyclic graphs with explicit dependencies [1]. GitPulse implements the same dependency order in a single function chain triggered by one user action.

A. CURRENT CONTROL FLOW

The order is fixed: ingest, save raw files, transform, delete the repository's old rows, load the new rows, commit, clear caches, rerun the dashboard. Exceptions are caught at the dashboard level, displayed to the user and stop further rendering. The transactional load means a failure never leaves a half-written repository.

B. CONFIGURATION AND REPRODUCIBILITY

The dependency set is declared in a requirements file (requests, python-dotenv, pandas, numpy, streamlit, plotly), and credentials are externalized to the environment. Together with the saved raw JSON, this allows an analysis to be reproduced on another machine.

C. MOVING TOWARD MANAGED ORCHESTRATION

Two changes would bring GitPulse in line with the practices in [1]. A scheduler would refresh tracked repositories periodically instead of only on demand, and each stage would become an independent, retryable task with its own logs. Section XII discusses these directions.

IX. EVALUATION APPROACH

A health score is useful only if it behaves sensibly, so we define an evaluation protocol along four axes. We report the protocol rather than numeric results, since the quality of the score depends on the repositories chosen and the run date.

TABLE 5. Evaluation protocol for GitPulse.

Axis	Question	Method
Correctness	Do metrics match the source?	Compare counts and closing times for sampled repositories against the GitHub web interface
Discrimination	Does the score separate active from abandoned projects?	Analyze a set of actively maintained, slowing and archived repositories and check the ranking
Sensitivity	How stable is the score under different weights and references?	Vary weights and reference values and record rank changes between repositories
Cost	What does one analysis cost?	Record API calls, runtime and database size per repository

Two expected behaviors follow directly from the design. First, because ingestion is capped at 300 records per endpoint, the commit-frequency component saturates for very active repositories: 300 commits over about 13 weeks is roughly 23 per week, which already exceeds the reference of 20. Second, the sensitivity analysis is important because the score is a weighted sum, so a change of reference value can reorder close repositories. Reporting these checks alongside the score is part of using it responsibly.

X. GAP ANALYSIS

Reading the implementation critically reveals ten gaps between the current design and a dependable repository analytics platform; Table 6 summarizes them by pipeline stage.

TABLE 6. Summary of the gap analysis.

Gap	Stage	Description	Consequence
(i) Capped sampling	Ingestion	At most 3 pages of 100 records per endpoint	Active repositories are under-counted; commit-frequency score saturates
(ii) Identity ambiguity	Transformation	Commit authors are git names, PR and issue authors are logins	The same person can be counted twice, or two people merged, in contributor counts
(iii) Closed versus merged	Metrics	PR closing time counts any closed PR, merged or not	Rejected or abandoned PRs distort responsiveness
(iv) Key collision	Storage	Commit primary key is the hash alone	Forks or repeated hashes across repositories can break loading
(v) Full refresh	Storage	All rows for a repository are deleted and reloaded	Wasted API calls; no history of how the score evolved

Gap	Stage	Description	Consequence
(vi) Window inconsistency	Metrics	Commits are fetched for 90 days, but the dashboard headline uses 30 days and the weekly series uses all stored commits	Users may confuse the windows unless the labels are precise
(vii) Open-state bias	Metrics	Resolution time uses only closed items	Long-open items are invisible, flattering repositories with large stale backlogs
(viii) Fixed references	Health model	Reference values are constants (20, 15, 72 h, 168 h)	Scores are not comparable across repositories of very different scale
(ix) No automated tests	Whole pipeline	No unit or integration tests of transformation and metric functions	Regressions go unnoticed
(x) Single-user storage	Storage	SQLite with no concurrency	Not suitable as a shared service

XI. CHALLENGES

Beyond the gaps in the implementation, several challenges are inherent to repository analytics.

- **(i) API limits and cost.** The authenticated REST API enforces rate limits, and search endpoints are limited more tightly. Any approach that analyzes many repositories must budget calls, cache responses and use conditional requests.
- **(ii) Data quality.** Repositories include bots, mirrors, personal projects and automated commits, which inflate activity [4]. Distinguishing human from automated contributors is an open problem.
- **(iii) Metric validity.** Activity is only a proxy for health. A mature, stable library may have few commits and still be healthy, while a hyperactive project may be unhealthy. A single score cannot capture this and should be complemented by context [9].
- **(iv) Gaming and fairness.** Any published score creates an incentive to optimize it, for example by splitting commits or closing issues quickly without resolving them.
- **(v) Reference calibration.** Choosing the values at which activity counts as full marks is subjective and domain dependent.
- **(vi) Privacy and ethics.** Commit authorship contains names and sometimes emails. Aggregating contributor data into published rankings raises questions of consent and proportionality.
- **(vii) Reproducibility.** The API reflects the current state, so rerunning an analysis later can give different numbers for the same historical window when items are edited, transferred or deleted.
- **(viii) Technical debt.** As features accumulate, glue code and undocumented assumptions grow [15]. Keeping stages small and explicit is the main defence.

XII. FUTURE DIRECTIONS AND ROAD AHEAD

The gaps above point to a clear evolution path, which we recommend carrying out incrementally, one stage at a time, in the spirit of starting with a small end-to-end pipeline and extending it through iterations [1].

1. **Incremental ingestion.** Store the timestamp of the last successful run per repository, use conditional requests and the `since` parameter, and upsert instead of delete-and-reload. This reduces API cost and enables time series of the score.
2. **Complete pagination.** Replace the page cap with a time-bounded stop condition, supported by a rate-limit-aware retry strategy, so that sampling no longer depends on page counts.
3. **Identity resolution.** Use the GitHub login attached to each commit, when available, and a mapping table for the remaining names, so that all three data streams share one contributor identity.
4. **Richer PR metrics.** Record merge status and merge time, first-review latency and review count, and report merged-PR time separately from closed-PR time.
5. **Stale-backlog metrics.** Add the age distribution of items still open, so long-unresolved items count against the score.
6. **Adaptive normalization.** Calibrate reference values from percentiles of a reference population of repositories rather than fixed constants, and report the score as a percentile alongside the absolute value.
7. **Quality assurance.** Add unit tests for transformation and metric functions, data validation checks on every load (for example, closed dates not before created dates) and a small set of golden repositories as regression tests.
8. **Managed orchestration.** Move scheduled refreshes to a workflow engine, with retries, logging and alerting, and replace SQLite with a server database if multi-user access is required.
9. **Predictive analytics.** With a history of scores, move from descriptive to diagnostic and predictive analytics, for example forecasting maintainer attrition or early signs of abandonment [1].

XIII. CONCLUSION

Repository health measurement is, first of all, a data engineering problem, and a small, modular pipeline can solve it clearly enough to teach from. In this tutorial paper we described GitPulse stage by stage, from the GitHub API connection and paginated ingestion, through transformation, relational storage and metrics, to an explainable 0 to 100 Health Score and an interactive dashboard. We formalized the score with reference-value normalization and automatic weight renormalization, and we showed with a worked example how it behaves when data are missing.

Our critical analysis also shows that the current design has limits: capped sampling, ambiguous contributor identity, full-refresh loading and fixed calibration constants. Addressing them through incremental ingestion, identity resolution, richer PR and backlog metrics, adaptive normalization, automated validation and managed orchestration would turn GitPulse from a single-user analytics tool into a reliable repository-intelligence platform. We hope the stage-by-stage view helps students and practitioners apply the same principles to other data sources.

REFERENCES

[1] E. Zeydan and J. Mangues-Bafalluy, "Recent advances in data engineering for networking," *IEEE Access*, vol. 10, pp. 34449–34496, 2022, doi: 10.1109/ACCESS.2022.3162863.

[2] GitHub, Inc. *GitHub REST API Documentation*. Accessed: Oct. 2026. [Online]. Available: https://docs.github.com/en/rest

[3] G. Gousios, "The GHTorrent dataset and tool suite," in *Proc. 10th Work. Conf. Mining Softw. Repositories (MSR)*, May 2013, pp. 233–236.

[4] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proc. 11th Work. Conf. Mining Softw. Repositories (MSR)*, May 2014, pp. 92–101.

[5] CHAOSS Project. *Community Health Analytics in Open Source Software: Metrics and Models*. Accessed: Oct. 2026. [Online]. Available: https://chaoss.community

[6] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61.

[7] SQLite Consortium. *SQLite Documentation*. Accessed: Oct. 2026. [Online]. Available: https://www.sqlite.org

[8] Streamlit Inc. *Streamlit Documentation*. Accessed: Oct. 2026. [Online]. Available: https://docs.streamlit.io

[9] S. Jansen, "Measuring the health of open source software ecosystems: Beyond the scope of project health," *Inf. Softw. Technol.*, vol. 56, no. 11, pp. 1508–1519, Nov. 2014.

[10] H. Borges and M. T. Valente, "What's in a GitHub star? Understanding repository starring practices in a social coding platform," *J. Syst. Softw.*, vol. 146, pp. 112–129, Dec. 2018.

[11] J. Coelho and M. T. Valente, "Why modern open source projects fail," in *Proc. 11th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*, Sep. 2017, pp. 186–196.

[12] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[13] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," *Bull. IEEE Comput. Soc. Tech. Committee Data Eng.*, vol. 36, no. 4, 2015.

[14] N. Garg, *Apache Kafka*. Birmingham, U.K.: Packt Publishing, 2013.

[15] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2503–2511.

[16] Plotly Technologies Inc. *Plotly Python Open Source Graphing Library*. Accessed: Oct. 2026. [Online]. Available: https://plotly.com/python/

[17] T. Akidau, S. Chernyak, and R. Lax, *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. Sebastopol, CA, USA: O'Reilly Media, 2018.